

1. Create a simple Node.js Express app that displays 'Welcome to My Playlist' at the root route, then write a Dockerfile to containerize this app and build the image locally.

ANS

- I created a simple Node.js Express application that displays "**Welcome to My Playlist**" at the root route.

- **Project Structure**

playlist-app/

└─ app.js

└─ package.json

└─ Dockerfile

app.js

- past a script file
- COPY package*.json ./
 - RUN npm install
- COPY app.js .
 - EXPOSE 3000
- **Build Docker Image**
 - docker build -t playlist-app:latest .
- **Run the Container Locally**
 - docker run -d -p 3000:3000 --name playlist-container playlist-app:latest

2. Push your container image to Docker Hub using your own account and tag it as playlist-app:latest.

***Hint:** Use 'docker tag' and 'docker push' commands after logging in to Docker Hub.*

ANS

- First, I logged in to Docker Hub:
- docker login
- I tagged the image with my Docker Hub username:
 - docker tag playlist-app:latest
YOUR_DOCKERHUB_USERNAME/playlist-app:latest
- I pushed the image to Docker Hub:
- docker push YOUR_DOCKERHUB_USERNAME/playlist-app:latest
- To verify the image:
- docker images
- The Docker Hub repository will contain:

3. Set up a GitHub Actions workflow (YAML file) in your repository to automatically build and push the Docker image to Docker Hub whenever you push to the main branch.

ANS

- I created the following file in my GitHub repository:
 - .github/workflows/docker-build-push.yml
- The workflow automatically builds and pushes the Docker image whenever code is pushed to the main branch.
 - name: Build and Push Docker Image
- **GitHub Secrets**
 - I added the following secrets:
 - DOCKERHUB_USERNAME
 - DOCKERHUB_TOKEN
- The Docker Hub access token is stored in GitHub Secrets instead of putting credentials directly into the workflow.

4. Modify your GitHub Actions workflow to also deploy the updated container image to a Kubernetes cluster using kubectl apply, referencing a deployment YAML file in your repo.

Hint: Use the 'kubectl set image' command to update the deployment with the new image tag.

ANS

- I created a Kubernetes Deployment YAML file named:
 - playlist-deployment.yaml
 - **playlist-deployment.yaml**
 - apiVersion: apps/v1
 - kind: Deployment
 - metadata:
 - name: playlist-app
 - spec:
 - replicas: 2
 - selector:
 - matchLabels:
 - app: playlist-app
 - template:
 - metadata:
 - labels:
 - app: playlist-app
 - spec:
 - containers:
 - name: playlist-app

- image: YOUR_DOCKERHUB_USERNAME/playlist-app:latest
 - ports:
 - containerPort: 3000
 - The deployment can be created manually with:
 - kubectl apply -f playlist-deployment.yaml
 - Check the deployment:
 - kubectl get deployments
 - run: |
 - kubectl apply -f playlist-deployment.yaml
 - kubectl set image deployment/playlist-app \
 - playlist-app=\${ secrets.DOCKERHUB_USERNAME }}/playlist-app:latest
 - name: Check Rollout
 - run: |
 - kubectl rollout status deployment/playlist-app
5. Use ChatGPT or GitHub Copilot to review your CI/CD workflow YAML and suggest at least two improvements for reliability or security. List the suggestions and explain how you would implement them.

ANS

- I used ChatGPT to review the CI/CD workflow and identified the following improvements.
- **Improvement 1: Use Versioned Docker Image Tags**
- **Problem:** Using only the latest tag makes it difficult to identify which version of the application is deployed.
- **Implementation:**
- Then deploy the specific version:
 - kubectl set image deployment/playlist-app \
 - playlist-app=YOUR_DOCKERHUB_USERNAME/playlist-app:\${GITHUB_SHA}
- This makes deployments traceable and easier to roll back.
- **Additional Reliability Improvement: Verify Deployment**
 - I would also add:
 - name: Check Kubernetes Rollout
 - run: kubectl rollout status deployment/playlist-app --timeout=120s
- This makes the pipeline fail if the Kubernetes deployment does not become ready instead of reporting a successful deployment prematurely.